Programming Language Design

Module 10: Typeclasses

Last module

We learned about typeclasses.

We saw some examples, espeially `Num`.

[What defines a typeclass? That is, what things separate one typeclass from another?]

This module

We're going to learn some *very important typeclasses*

Specifically:

1. `Monoid`
2. `Functor`
3. `Applicative`

At first, these will seem like very abstract things.

It turns out, you already know the first two pretty well, but you don't realize it yet.

The third one will take some understanding.

When Haskell gets weird

Haskell has a reputation. You're probably starting to understand that now.

What we're going to cover is precisely the point where Haskell gets *weird*. That is, where it starts using weird terms from category theory for everything.

Rest assured: the weird names like `Monoid` refer to perfectly ordinary things that occur naturally in math and in other programming languages.

You don't need to be a category theorist to understand Haskell. (which is good or I wouldn't be qualified to teach this class)

Now for some math

Okay, it's time for some abstract math.

You see, mathematicians came up with this idea of typeclasses long before programmers ever did.

Consider the idea of a Semigroup.

This is a concept from mathematics. A semigroup is a set (i.e., a datatype) that also has an associative binary operation that it is closed under.

For example, integers are a semigroup "under" addition, because the sum of two integers is another integer. (we use the preposition "under" to describe what the unifying operation is because sometimes there's more than one: like multiplication)

Can you come up with more?

What about integers under multiplication? Yes, that is also a semigroup!

To have a semigroup, you just need a set (or a datatype) and a binary function with type `ElementOfThatSet -> ElementOfThatSet -> ElementOfThatSet`

[Can you come up with some more examples?]

Why abstract structures?

Mathematicians invented abstract structures like this because they can prove things about entire families of sets and operations.

For example, Fields are kind of like semigroups, but they have two operations instead of just 1, and they have two identities (1 for each op.). Also both operations are invertable.

Did you know that if a Field's set is finite, then its size has to be a power of a prime number?

Why? Because it turns out if that weren't the case, the second operation would not always be invertible. This fact comes from number theory. It's not an axiom that the size has to be a power of a prime, it just naturally falls out of the rules.

Semigroups are in Haskell

It might seem strange, but Semigroups are in Haskell too.

They're actually very useful.

So far we've learned a lot of Haskell and we've tried to make analogies to other programming languages. However, that had to end at some point. This is something most languages do not have (but maybe should).

So what is a Semigroup? Let's look at its typeclass and see if we can figure it out...

The `Semigroup` class

```
class Semigroup a where
    (<>) :: a -> a -> a
    sconcat :: NonEmpty a -> a
    stimes :: Integral b => b -> a -> a
```

A semigroup is any mathematical structure (read: datatype) that has a closed binary operation. That's the point of `<>` above.

This isn't a standard mathematical operator with a well-known meaning. It's a placeholder. You can define it however you want, and different semigroups define it completely differently. Sometimes it's `+`, sometimes it's `++`, etc.

The other two functions (`sconcat` and `stimes`) are actually optional. There is a *default definition* in terms of `<>`, so you actually only need to define `<>`. Let's see some examples of `<>` first, and then talk about the other two functions.

A really basic semigroup

Remember when I said that integers form a semigroup *under* sum?

That is, if I add two integers together, I get another integer. That's a binary operation.

Therefore, integers and addition are a semigroup.

Unfortunately, integers and multiplication are also a semigroup. There is more than one option for a reasonable semigroup with integers.

So, how do we distinguish them?

Special data types

There is a `Sum a` data type. So `Sum Integer`, `Sum Float`, etc.

The only requirement is that `a` be a number.

This data type implements `Monoid`.

So, now, we can, uh, add numbers together...

```
import Data.Semigroup
Sum 10 <> Sum 20 == Sum {getSum = 30}
```

(the `Sum` data type stores the actual integer inside of a field named `getSum`, I'm guessing because it means they didn't need to add a pattern matcher)

Isn't that handy? We can add numbers everyone! That's super useful right!

I know what you're thinking

Okay! Give me a second! I know what you're thinking. "We could already do that! And it was easier! And we didn't need to import `Data.Semigroup` or use `getSum` to get the result back out!"

Okay, true. The real point behind `Semigroup` isn't that it lets you add numbers. It's the other two functions. What were they again?

```
class Semigroup a where
    (<>) :: a -> a -> a
    sconcat :: NonEmpty a -> a
    stimes :: Integral b => b -> a -> a
```

Right, `sconcat` and `stimes`.

The `sconcat` function

`sconcat` will take any `NonEmpty` list of a monoid, and convert it into a single value.

That's the real power of `Monoid`s. They represent a universal "foldable" type in which we don't have to specify associativity.

Remember with folding we needed to specify `foldl`, `foldr`, `foldl'`, etc.? We had to specify the associativity and strictness, because it was a meaningful distinction.

Sometimes it's not meaningful, and if something is a Semigroup, we can just flatten any non-empty list into a single thing.

And what's more, we don't have to ask how to do it. We know how to do it. `Sum` is flattened with `+`. So `sconcat` is less work to call than `fold` et al.

So instead of `Semigroup`, think "naturally foldable without extra info"

Questions?

What is `NonEmpty`?

Which brings us to `NonEmpty`. What is the point?

`NonEmpty a` is basically a `[a]`, except that it has no empty constructor.

A `List a` has two constructors: `[]` and `a : [a]`.

That is, we can either create an empty list or add the value to an existing list.

`NonEmpty` only has one constructor: `a :| [a]`
The `:|` is just like `:` but for `NonEmpty`. It means we must have an existing list (which can be empty) *and* a value to construct a `NonEmpty`.

Therefore, it is impossible for a `NonEmpty` to ever be empty, because we needed at least one value to construct it.

And when we destructure it, the resulting right side list might be empty, which is right.

Using `sconcat`

We can use `sconcat` to sum some values:

```
sconcat $ Sum 10 :| [Sum 20] == Sum {getSum = 30}
```

We can use the same function to multiply too:

```
sconcat $ Product 10 :| [Product 20, Product 30] == Product {getProduct = 6000}
```

And if we get tired of writing `Product`, we can use `map`, right? Not quite, we haven't learned `Functor` yet, and `map` only works for regular lists, not `NonEmpty` (we can do this though: `sconcat $ fmap Product $ 10 :| [20, 30]`)

We can also use `sconcat` to concatenate strings, without any extra work:

```
sconcat $ "hey" :| ["there", "hi", "there"] == "heytherehithere"
```

Why `NonEmpty`?

So why do we have to use this janky weird list instead of a regular list? Why can't `sconcat` just take a normal list? Who cares if the list is empty?

Because `Semigroup` doesn't have a "default" value. What should the sum of an empty list be? You might assume it would be `0`, and that makes sense, but what about the product? In that case, it probably makes more sense for it to be `1` (because `1` means the same thing as "do nothing" when multiplying)

Because it's not obvious, `Semigroup` doesn't make a distinction. Not in Haskell and not in mathematics: semigroups in math are not required to have an identity element.

And that's annoying, because if we had an identity element, we'd have a useful value to use if the list were empty, and then we wouldn't have to use `NonEmpty`.

Monoid

A semigroup which also has an identity element is called a *Monoid*, both in Haskell and in general mathematics.

There is a `Monoid` typeclass just like you'd expect:

```
class Semigroup a => Monoid a where
    mempty :: a
    mappend :: a -> a -> a
    mappend = (<>) -- by default, `mappend` is just the Semigroup operator
                   -- don't override it: that makes sense.
    mconcat :: [a] -> a
```

Typeclasses can require a global value

First, this illustrates a cool thing about typeclasses in Haskell.

Notice that `mempty` is a constant. The typeclass can say "there needs to be a value named `mempty` somewhere".

This is a rare and powerful feature. In Java, we can't use inheritance to require the existence of a value (only of a method, called on an object, that produces a value).

Also, now that we have an `mempty`, we don't need the list to be `NonEmpty`. Now if the list is empty we just return `mempty`.

What is `mempty` for some common values?

```
ghci> mempty :: Sum Int
Sum {getSum = 0}
ghci> mempty :: Product Int
Product {getProduct = 1}
ghci> mempty :: String
""
```

That last one is why it's called `mempty`: the empty string is a prime example.

It's an identity element of strings under concatination, because if you concatinate an empty string, it doesn't do anything.

What about `stimes`?

There was another `Semigroup` function: `stimes :: Integral b => b -> a -> a`

This function calls `<>` on the same value the given number of times.

So `stimes 3 monoid` is equivalent to `monoid <> monoid <> monoid`

So for the `Sum` monoid, it's the equivalent of mulitplication (hence the name). For `Product`, it's exponentiation.

For strings, it's particularly interesting: `stimes 5 "hi" == "hihihihihi"`
or more naturally: `5 `stimes` "hi" == "hihihihihi"`

Wrinkle: for `stimes 0 ...` to make sense, the `Semigroup` must be a `Monoid`. [why?]

Get that?

Because of the `Semigroup` typeclass, we automatically get the ability to repeat strings.

We didn't need a special operator for it that only works on strings. It actually works on them because they form a semigroup.

Notice the Haskell mindset: find a way to encode a mathematical structure. They often elegantly describe coding patterns.

Very cool: soon we will learn about monads, which are one way that Haskell represents commands to do IO. Some monads are also monoids, which allows us to use `stimes` on them. So `5 `stimes` putStrLn "hi"` actually prints "hi" 5 times like you would expect.

Summary of Semigroups

- `Semigroup` is a typeclass
- To make something a semigroup, it needs a closed binary operation (called `<>`)
- Once you make something a semigroup, you can call `sconcat` and `stimes` on it.
- `sconcat` requires a non-empty list, and `stimes` isn't guaranteed to work on `0`, because a `Semigroup` is not required to have an identity element.

Summary of Monoids

- `Monoid` is a typeclass which *inherits* from `Semigroup` (which means it must be a `Semigroup` first to be a `Monoid`).
- A `Monoid` must have an identity element, which is called `mempty`.
- The operation `<>` is also called `mappend` (this is because `Semigroup` came later)
- `mconcat` is like `sconcat`, but now the list can be empty.
- `Semigroups` represents operations that are *collapsable* and repeatable. They represent operations that can be squashed down into a single value, or that can be done over and over.

Questions?

Semigroup laws (technically law)

Just because we can provide an operator `<>` doesn't mean it's valid.

`Semigroup` instances also are supposed to follow certain laws (there's only one, technically, but most of these mathematical typeclasses have more than one)

These laws are not checked by Haskell. Instead, you must check your own code and make sure it follows these rules.

If it doesn't, weird, non-deterministic things can happen depending on which Haskell implementation you use.

You will also upset any Mathematicians nearby, which is risky.

Note: the type checker can make sure that, e.g., the operator is a binary operator with type `a -> a -> a`. But some laws are not visible to it.

Semigroup laws (technically law) (2)

The one law that every semigroup must follow: the operation `<>` is associative.

That means: `(a <> b) <> c == a <> (b <> c)` for *any* instances of your semigroup.

Addition, multiplication, and string concatenation all follow this rule.

But, e.g., if `<>` meant "midpoint", it wouldn't work. `(1 <> 2) <> 3 == 1.5 <> 3 == 2.25`, but `1 <> (2 <> 3) == 1 <> 2.5 == 1.75`. So the real numbers under the midpoint operation is not a semigroup.

Why do we care, though? Because, fundamentally, `Semigroups` are things that are naturally "timesable". That is, they have an operation that behaves like addition, and that can be extended into multiplication by repeatedly doing them an integral number of times.

Semigroup law(s) (3)

Also unlike `fold`, `sconcat` does not specify whether it goes right to left or left to right.

In fact, it might even be interleaved on special hardware (e.g., SIMD).

This additional flexibility is nice, but it means some operations just aren't semigroups.

Monoid laws

Monoids have laws too.

First, all monoids are semigroups, so they follow the semigroup law (`<>` is associative).

In addition, recall that monoids have an indentity element `mempty :: a`. Note: nothing about the type requires it to actually *be* an identity element, but if it's not, get ready for weird bugs.

To make it an identity, this must follow:
`x <> mempty == mempty <> x == x` for any instance `x` of the monoid.

Laws in general

Many of these mathematically-based typeclasses have laws to follow.

You are responsible for following them.

In languages like Idris, the language actually treats proofs as a first-class value. In order to make something a Monoid, you can actually have the typeclass require proofs be supplied that it obeys associativity, left identity, and right identity.

The type system is *dependently typed*. Dependently typed type systems can actually encode logical predicates, and values of those types are proofs.

Haskell has some community members who want to add this feature, but it's not there yet, so until then, you need to take note when a typeclass has *laws*. The type system won't protect you, here.

Questions?

Making a monoid

To make sure we understand this concept, let's make a monoid of our own.

Consider the `max` function on unsigned integers. Haskell calls an unsigned `Int` a `Word`. Let's define a `newtype` to make into a `Monoid`.

```
newtype Max = Max Word deriving Show -- deriving Show for convenience
```

Now, we need a new type because `Word` can't be a monoid by itself. There are more than one useful operation (addition, multiplication, etc.) that we could define on a `Word`. Therefore, we are creating a new data type which will only work with one operation.

`Max` is a type, but also a constructor. It stores a single word.

Making a semigroup

Not so fast. In order for something to be a monoid, it needs to be a Semigroup, first.

Let's make taking the maximum between two `Word`s our `<>` operation:

```
instance Semigroup Max where
    (Max x) <> (Max y) = Max $ max x y
```

So now, if I write `Max 20 <> Max 30` I get `Max 30`, which makes sense.

What about the laws

Just because we created a typeclass instance doesn't mean it's valid. It needs to follow the law for `Semigroup` too, which requires that `max` over `Word` be associative.

Here's a simple proof by case analysis over permutations of `x`, `y`, and `z`.

```
forall natural numbers x, y, z, (x `max` y) `max` z == x `max` (y `max` z)
case x <= y <= z: both sides simplify to z
case x <= z <= y: both sides simplify to y
case y <= x <= z: both sides simplify to z
case z <= x <= y: both sides simplify to y
case y <= z <= x: both sides simplify to x
case z <= y <= x: both sides simplify to x
```

So we can make `Max` into a semigroup safely.

Making it a monoid

Of course, a `Semigroup` is fine, but a `Monoid` is much easier to work with.

Luckily, once a type is a `Semigroup`, we only need an instance of `mempty` to make it a `Monoid`.

What would be a good choice of `mempty`? That is, what is a value, where, if we take `max mempty x` we get `x` for all `x`?

Try 0

0 is a logical choice. `max 0 x` is always `x` for unsigned integers, and `max x 0` is also always `x`, so 0 is both a left and right identity.

```
instance Monoid Max where
    mempty = Max 0
```

Now that `Max` is a monoid, we can take the maximum of a list of numbers instead of only applying it between pairs:

`print $ mconcat $ map Max [1, 0, 50, 2, 30, 4, 9, 0]` prints `Max 50`

Knowledge check: what about Min?

Could we make `newtype Min = Min Word` a `Semigroup`? What about a `Monoid`?

Answer: sort of

We can make it a `Semigroup`. We just replace `max` with `min`.

Making it a `Monoid` is harder. We can't use 0 as our identity element. Instead, we would need to use maxBound. The biggest number is the only one that will never be the minimum with a less-big number.

If we used an indefinitely large datatype, like `Integer`, then there is no identity element. Therefore, in that case, we couldn't make it a `Monoid`, and every list of elements to take the min over would have to be non-empty for a meaningful result.

I have no idea what's going on

While you will need to go back, come to office hours, and/or do some studying, I will provide this super-short summary of what we've discussed so far to help guide your own work. These bullet points are roughly in order of when we learned them, so you can stop as soon as you hit a confusing one and know what to review.

- A data type is basically a combination of a struct and/or an enum. They can have multiple constructors (like an enum has variants), but they can also have multiple fields like a struct. The `newtype` keyword is preferred if there is exactly one constructor with one field, the `data` keyword is used otherwise.
- A typeclass is basically a collection of datatypes and the functions that they all support. It's like an OO interface. When a datatype is an *instance* of a typeclass, that's like *implementing* an interface. For example, because `Int`, `Word`, and `Integer` all are instances of the `Num` typeclass, we can call `+` on all of them.

I have no idea what's going on (2)

- `Semigroup` is a typeclass for any type that has a closed, associative, binary operation which we call `<>`.
- The real purpose of this operation is to run it over and over: either on a collection of things to combine them all down into one or a certain number of times.
- Sometimes we need to define a `newtype` or `data` to create a new semigroup, because there are more than one useful operation. (e.g., `Sum Int` and `Prod Int`)
- `Semigroup` is useful becuase of the `sconcat` method, which magically flattens any non-empty list. What does flatten mean? It means applying `<>` repeatedly.
- `stimes` just repeats `<>` the given number of times.
- `sconcat` requires a non-empty list, constructed with `value :| regularList`. It has to be non-empty because there isn't necessarily an identity element.

I have no idea what's going on (3)

- For example, `sconcat` on `Sum 10 :| [Sum 10, Sum 10]` yields `Sum 30` `sconcat` on `"hey" :| ["there", "hi", "there"]` yields `"heytherehithere"` `sconcat` on `Max 0 :| [Max 10, Max 20]` yields `Max 20`.
- But it's annoying to have to use `:|` to construct `NonEmpty` lists. Therefore, we have `Monoid`, which is a `Semigroup` that also has a "do nothing" element named `mempty`. Mathematicians know this as the identity element.
- Now we can use `mconcat` and pass a normal list instead of a `NonEmpty` list. If the list is empty, it returns `mempty`.

I have no idea what's going on (4)

- Fundamentally, a `Semigroup` or `Monoid` represents a kind of thing that can always be combined with others of the same sort to simplify a collection into one value.
- It also represents a kind of thing that can be combined with itself a certain number of times. That's what `stimes` does.
- Even `IO` code in Haskell is a `Monoid`: `stimes 5 $ putStrLn "hi"` prints hi 5 times.
- There are lots of examples of where this is useful:
  - Matrices are monoids: matrix multiplication combines two matrices into one.
  - Strings are monoids: string concatenation is `<>`
  - Game events in a video game could be monoids
- Most languages make you treat all these cases different. Haskell lets you just `mconcat` them all down into one value automatically, using `<>`.

Using that summary

If something stood out to you in those slides, please ask about it.

We're about to learn another abstract typeclass from math, so it's a good time to clarify.

One of the things that makes abstract stuff hard is that "monoid" is such a general concept that it feels like it could apply to tons of different things, but that's also why it's so useful. Feel free to ask for more examples.

Questions?

Functors

You know what's a useful function? `map`.

It lets you take a unary function, and a list, and apply the function to every element of the list! What's not to like?

Hey, I like the list `[1, 2, 3]`, but I'd like it more if every element were one larger.

Boom: `map (+1) [1, 2, 3] == [2, 3, 4]`

Wow! Thanks, `map`!

Functors (2)

But you know what's lame? I can only map things over a list.

What if I have a tree?

```
data Tree a = Leaf a | Inner [Tree a] deriving Show
```

This kind of tree is either a `Leaf`, or a list of trees.

We can represent any kind of tree with this, but you know what we can't do? `map`!

What if we want to apply a function to every item in the tree?

We can map the function over the list in the `Inner` constructor, but what if the tree is a `Leaf`? And also, we want it to propagate further down.

The Functor class

This is what a `Functor` is:

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
    (<$) :: a -> f b -> f a -- the "replace" operator. optional.
```

The only required function a `Functor` has is `fmap`. That just means "functor map". That is, a `Functor` is any type that has a `map` implementation, and we call it `fmap`.

There is an interesting wrinkle to this type class, though. Can anyone see what it is?

Consider how `f` is being used in the types of `fmap` and `<$`

Higher kinded types

```
class Functor f where
    fmap :: (a -> b) -> f a -> f b
    (<$) :: a -> f b -> f a -- the "replace" operator. optional.
```

Notice that we're writing `f a`. `f` isn't being used by itself: it takes an argument.

However, `f` is a type. This is the first time we have seen a very rare and powerful Haskell feature: the higher-kinded type.

Before going into that, let's make sure this typeclass definition makes sense. `fmap` is a function that takes a function from `a` to `b`, and a functor of `a`s, and it returns a functor of `b`s.

Remember, list is a functor, so just imagine how `map` works. It takes a function `a -> b` and a list of `a` and it produces a list of `b`. Same idea but it works for any functor.

Type Constructor

Okay, but what was that about higher-kinded types? What does that mean?

A *kind* in general is word that we use to describe a type, or maybe a type constructor.

What is a type constructor? A type that takes a type:

```
Pair a = Pair a a
```

This type, `Pair`, takes a type. It's kind of like a function.

In fact, it's like a constructor for a new type. `Pair` takes a type, like `Int`, and constructs a new type: `Pair Int`.

And that's why it's called a type constructor.

Higher kinded types (2)

Is a type constructor a higher kinded type? Well, the type which *has* one is.

But not every type constructor is the same.

Consider a simple datatype, like `Int`. It's *kind* is `*` (pronounced "splat")

Consider a list. Lists are different from `Int`s. You can't just make a list without knowing what it's a list of. So `[]` is like a function, it's a function that takes a type and returns a more specific kind of list. So its kind is `* -> *`.

That means "`List` takes a specific type like `Int`, and it returns another specific type `[Int]`. So `List` is like a function that takes a type and returns a type. `* -> *`.

If you want to know the kind of a type, you can use `:k` in `ghci`. E.g., `:k []` prints `[] :: * -> *`

Higher kinded types (3)

In general, the kind of a type is how many other types you need to know to make one.

For example, an `Int` is self contained, and its kind is `*`.

A list requires one extra piece of information: `* -> *`.

What about `Maybe`? Also `* -> *`. It can be a `Maybe Int` or a `Maybe String`.

What about `Either`? That one is a `* -> * -> *`. It requires *two* types, one for `Left` and one for `Right`.

Higher kinded types (3)

As we saw in the `Functor` type class, we named the functor `f`, and we were applying it: `f a`.

That means "some functor of `a`s".

This means a functor *must* be a higher kinded type. It's always a functor *of* something.

List is a functor, and list meets this constraint. We always have a list of something.

Haskell is one of the rare programming languages that has a higher kinded type system. Many languages have parameterized types (like `ArrayList<T>` in Java), but you can't really say "this is an interface that takes a collection with type `a` in it and returns the same type of collection with type `b` in it".

Questions?

The replace operator `<$`

Recall that to be a functor, a type constructor must have the `fmap` function, which is supposed to apply a given function to values inside the functor.

However, there's also an optional `<$` operator that they can support. This is the "replace" operator. It replaces whatever data is in the functor with the first argument.

Here's an example of how `<$ :: Functor f => a -> f b -> f a` works:

```
ghci> 20 <$ [10, 20, 30]
[20, 20, 20]
```

It's an operator that is useful to "project" a value into many places, but I don't find myself using it often. [Can you write a default implementation of `<$` in terms of `fmap`?]

The fmap operator `<$>`

Functors are so common, there is a special operator for them: `<$>`. It's just a synonym for `fmap` but it's written infix:

```
fmap (+1) [1, 2, 3] == [2, 3, 4]
(+1) <$> [1, 2, 3] == [2, 3, 4]
```

Honestly I think I see `<$>` more often than `fmap`, so maybe they should have made that the way the typeclass is defined.

More functors

Okay, list is a functor, we get it. But what are some other functors?

`Maybe` is a useful functor. Does it have the right kind? Yes, `Maybe` is an "of" type, it has kind `* -> *`. That is, a `Maybe Int` is a "maybe of ints".

What happens when we use `<$>` on a Maybe? It depends:

```
(+1) <$> Just 1 == Just 2
(+1) <$> Nothing == Nothing
```

A Maybe is kind of like a special list that can only have zero or one elemnt, so this behavior makes sense. If we call `fmap` on an empty list, we get `[]`. If we call `fmap` on a `Nothing`, we just get `Nothing`.

But if the `Maybe` is a `Just`, we apply the function to the value inside the `Just`.

What about Either?

It doesn't make sense to make `Either` a functor, right?

If I have an `Either String Int`, if I apply `(+1)` to it, what should happen?

If it's a `Right`, sure, add one to it. But what if it's a `Left`? We can't `(+1)` a string!

It also doesn't have the right kind!
Functors are supposed to be `* -> *`. But either is a `* -> * -> *`.

And it's true, `Either` is not a functor...but `Either a` is!

Wait, what's the difference between `Either` and `Either a`?

What about Either? (2)

`Either` is a type with kind `* -> * -> *`

`Either a` is a type in which we have already decided that the left type will be `a`, and we're just waiting for the right type. Therefore, `Either a` has kind `* -> *`.

Proof:

```
ghci> :k Either
Either :: * -> * -> *
ghci> :k Either Int
Either Int :: * -> *
```

And it turns out, `Either a` *is* a functor!

What about `Either a`?

```
data Either a b = Left a | Right b
    deriving Show

instance Functor (Either a) where
    fmap f (Left x) = Left x
    fmap f (Right y) = Right $ f y
```

With `Either`, `fmap` only applies when it's a `Right`. If it's a `Left` the map is ignored.

If it's a `Right`, we apply the function to the value.

Why? Because `Left` represents the "an error occurred" case. Once we enter that case, we don't do any further computation.

We therefore have a useful way to apply functions as long as a value is valid.

What about `Either a`? (2)

So, `Either` is not a functor, but `Either a` is. For any `a`.

What about `Either b`? That's the same thing. The `b` just represents a type.

`fmap` has the type `Functor f => (a -> b) -> f a -> f b`

In this case, if we consider `Either String` to be our functor, `f a` means `Either String a`. So `fmap` in the context of `Either String` has type:

```
(a -> b) -> Either String a -> Either String b
```

The logic here is that the function modifies only right values, so the right type changes because of the function, but the left type (in this case `String`) stays the same.

Questions?

Functor knowledge check

1. Create a new type called `Either3 a b c` which has three constructors, `Leftmost a`, `Middle b`, and `Rightmost c`.
2. What is the kind of this type?
3. Make `Either3 a b` a functor. Apply `fmap` only if it is a `Rightmost`.
4. Could we also make `Either3 a` a functor? If so, what would `fmap` look like? If not, why not?
5. What would `(+1) <$> Rightmost 20` return? What about `Middle 20`? Assume the type is `Either3 Int Int Int`

Functor KC answers

```haskell
-- 1.
data Either3 a b c = Leftmost a | Middle b | Rightmost c
-- 2. The kind is: * -> * -> * -> *
-- 3.
instance Functor (Either3 a b) where
    fmap f (Rightmost x) = Rightmost $ f x
    fmap f (Leftmost y) = Leftmost y
    fmap f (Middle z) = Middle z
    -- we can't just handle the other two cases with
    -- fmap f other = other
    -- because "other" could have a different `c` than the (Rightmost x).
```

4. No we could not. `Either3 a` has kind `* -> * -> *`, so it cannot be a `Functor`.
5. `(+1) <$> (Rightmost 20 :: Either3 Int Int Int) == Rightmost 21`

   `(+1) <$> (Middle 20 :: Either3 Int Int Int) == Middle 20`

Questions?

Functor laws

Remember how `Semigroup`s and `Monoid`s had laws they had to follow to make sure their behavior was predictable to the user?

Functors have laws too. Two of them:

1. `fmap id = id`. That is, mapping the identity function inside the functor won't change anything. This law forces `fmap` to *only* apply the function `f`, and not to modify the functor in any other way.
2. `fmap (f . g) = fmap f . fmap g`. This law is how functors are thought of in category theory. It basically says you can't take a compound function and do something wacky based on the specific identity of that function. Like "if it happens to be (+1) . (*2) just output 42 instead of computing the value". Instead, the functor must respect composition.

Warning about functors

Functor is a concept that comes from a branch of math called *category theory*.

It is a very abstract branch of math that is used to prove things about mathematical structures, families of structures, and their compositions.

This mathematical basis is partly what makes Haskell what it is. It turns out that good engineering emerges from good mathematics, so designing the language from these first principles is often a good idea.

Unfortunately, the use of abstract math terms also makes them hard to understand, and people use the terms sometimes without knowing that they are also mathematical terms (or without knowing what the term means).

Warning about functors (2)

There is a *Functor* design pattern in the C++ community.

In C++, a functor is a basically a class that overloads the `()` operator, so instances of it can be called like a function.

You can see why the name was chosen: it sounds like "thing that does function stuff".

However, it has *nothing* to do with `Functor`s in math. Don't get confused when you look it up! *Our* functors are things that support *map*, they don't have to be functions!

But...is a function a functor?

`->` is not one, but `a -> ...` *is* one

Specifically, ((->) a) is a functor. That is, a function from `a` to...something.

It has this instance definition:

```
instance Functor ((->) a) where
    fmap = (.)
```

Mapping a function `f` to another function `g` means applying `f` to the result.

That is the same as `f . g`

This gives us a little more insight into functors. They are data types that can be "composed" with a function, just like a function can.

Composition really is fundamental to functional programming. It's how we build more complex programs. So it makes sense that functors are important.

What can't a functor do?

But functors are not infinitely powerful.

We have seen an example of when we want to apply a function to every value in a list: `(+1) <$> [1,2,3,4,5]` returns `[2,3,4,5,6]`. Easy.

But what if we want to apply a list of functions to a single value?
`[(+1), (*2), (^3)] <???> [2]`

Well, we can do it, but not as a functor. The issue is that mapping is for applying a function outside of a functor to everything inside it.

But what if the functions we want to apply are *already* in a functor? That is, we have a functor full of functions and we want to use them with another functor?

Applicative functors

This more powerful kind of functor is called an *applicative functor*.

Haskell calls the typeclass `Applicative` for short. It looks like this:

```
class Functor f => Applicative f where
    (<*>) :: f (a -> b) -> f a -> f b -- <*> is pronounced "app"
    pure :: a -> f a
```

Here, `<*>` is the mystery operator from before. It lets us take a functor which has a function inside of it, then another functor, and it applies the function inside the first functor to the raw data in the second one.

`pure` just injects data into the data structure. So `pure 2 :: [Int] == [2]`

Applicative functors (2)

Notice the `Functor f => Applicative f` in the typeclass. In order for `f` to be an applicative, it must already be a `Functor`.

That is, all `Applicative`s are `Functor`s, but not necessarily all `Functor`s are `Applicative`s (although most are)

Recall that functors basically represented some kind of data structure that we could "inject" a function into.

An applicative functor is a functor which has a natural way to combine a functor with a function and a functor with data.

Let's see how it works for lists...

Lists as applicative functors

Before we saw the example of wanting to apply many functions to the same value:

```
[(+1), (*2), (^3)] <???> [2]
```

It turns out, `<*>` is the magic operation:

```
[(+1), (*2), (^3)] <*> [2] == [3, 4, 8]
```

When would we need to do this?

It actually wasn't obvious: Applicative functors were invented in 2008 in this paper: Applicative programming with effects.

They don't come from Math originally (at least the name doesn't, they are technically a kind of functor called a *lax monoidal functor*, which is a functor that is also a monoid and which is not necessarily invertible).

What happens for many elements in the 2nd arg?

So we saw this example:

```
[(+1), (*2), (^3)] <*> [2] == [3, 4, 8]
```

Lots of operations happening on a single value. We get a list of results.

But what would this do?

```
[(+1), (*2), (^3)] <*> [1, 2, 3] == ???
```

Many elements in 2nd arg (2)

We get this:

```
[(+1), (*2), (^3)] <*> [1, 2, 3] == [2,3,4,2,4,6,1,8,27]
```

That is:

```
[1 + 1, 2 + 1, 3 + 1, 1 * 2, 2 * 2, 3 * 2, 1^3, 2^3, 3^3]
```

Notice that it's:

```
[1 + 1, 2 + 1, 3 + 1] ++ [1 * 2, 2 * 2, 3 * 2] ++ [1^3, 2^3, 3^3]
```

We apply the first operation to every value in the second list, then the second operation to every value in the second list, etc. It's every combination.

But why?

When list is useful applicatively

Sometimes we want to do a sequence of operations to a bunch of values.

Since it's impossible to avoid using graphical user interfaces to illustrate fancy design patterns, imagine that you have a bunch of GUI widgets, and you need to update, position, and draw them.

That might look like this:

```
[Update, Position, Draw] <*> [QuitButton, LoadButton, BackgroundImage, ...]
```

The result will be a list of every action applied to every button.

It's a simple trick, but pretty cool.

Are there other `Applicative`s?

Yes, and arguably more useful ones. Consider `Maybe`.

We learned that `Maybe a` was a value that was either `Just` an `a`, or instead `Nothing`.

When we map over a `Maybe`, if it's `Just`, we apply the function:

```
(+1) <$> Just 2 == Just 3
```

And if it's nothing we don't do anything:

```
(+1) <$> Nothing == Nothing
```

But what if we do this?

```
(+1) <*> Just 2 == ???
```

That doesn't work

Then we get a type error.

But *this*:

```
Just (+1) <*> Just 2 == Just 3
```

Does work.

Wait, what? We have `(+1)` insid the just? So it's a Maybe of a function?

Yes, we can store the function itself in the maybe and then use it as an applicative.

And `Nothing` works like you'd expect:

```
Nothing <*> Just 2 == Nothing
Just (+1) <*> Nothing == Nothing
```

But...why?

This raises a question: why on Earth would we do that?

It's actually useful.

Suppose we have some function that takes 2 arguments.
However, the arguments have to be computed, and the computation can fail.
If the computation fails, we want the whole result to be `Nothing`
But if the computation succeeds for *both* arguments, we want the function to go thru.

Let the binary function be `+`, and the computation that can fail be:

```
fallableComputation :: String -> Maybe Int
```

We can represent this as follows...

Why (2)

```
Just (+) <*> fallableComputation <*> fallableComputation
```

The first `Just` establishes how many arguments the function can have. In this case `+` takes two arguments, so there can be two `<*>` s.

Each one is a computation that can fail.

If any computation fails, the whole thing fails.

But if they both succeed, the whole thing succeeds.

And Either?

Same thing. `Either a` is an applicative. It chains together `Either`s until one of them becomes `Left`, then the whole thing is the value of the first `Left`.

But if the whole computation succeeds, the result is a `Right` that is the result of the computation.

```
Right (+) <*> Right 2 <*> Right 2 == Right 4
pure (+) <*> Right 2 <*> Right 2 == Right 4 -- `pure` for Either a is just Right
Right (+) <*> Right 2 <*> Left "error" == Left "error"
Right (+) <*> Left "error" <*> Right 2 == Left "error"
Left "error" <*> Left "hello" <*> Left "world" == Left "error"
```

(Note: pure is the constructor for all `Applicatives`, but for `Either a`, it just wraps the result in `Right`.)

What `Applicative`s are

Functors are data structures that can have functions "injected" into them.

Applicatives are data structures that can be "chained".

What happens when they are chained depends on the data structure:

- `[a]` makes a new list that pairs every element of the left list with every element of the right. It lets you compute combinations, and it models non-determinism.
- `Maybe` takes a function and feeds operands to it unless one of them is `Nothing`, in which case the whole thing is `Nothing`.
- `Either` takes a function and feeds operands to it unless one of them is `Left x`, in which case the whole thing is `Left x` (and only the first `Left` value is the result).

What `Applicative`s are

The concept `Applicative` is basically "structure that can hold functions and be chained with their arguments".

However, it leaves "what happens when you chain" up to you.

[Can you think of any "chainable structures" that you've encountered in your coding? Think about concrete things that you've built for your classes: game events, image effects, sound filters, web server responses, etc.]

`Applicative` laws

In order for a structure to be `Applicative`, it's supposed to follow certain laws to make sure it behaves as everyone expects. Same as `Functor`. Here they are:

- Identity: `pure id <*> v = v`. Applying the identity doesn't do anything weird.
- Composition: `pure (.) <*> u <*> v <*> w = u <*> (v <*> w)`. Same for the composition operator. here, `u` and `v` are functions that are already injected inside of applicatives.
- Homomorphism: `pure f <*> pure x = pure (f x)`. "pure" also shouldn't do anything to any other function or value, and `<*>` calls the function on the value.
- Interchange: `u <*> pure y = pure ($ y) <*> u`. This means that `<*>` can't do something that changes "apply to y" to mean something different when it's on the right hand side. Remember that `$` is the function application operator.

`liftA2`

Another function you see along with applicatives is `liftA2`. Technically, this is a function that is part of the typeclass, and it can be defined instead of `<*>`. Its type is:

```
liftA2 :: (a -> b -> c) -> f a -> f b -> f c
liftA2 f x y = f <$> x <*> y
```

This stands for "lift action 2-arg funciton". It just means "take a function and apply it to the values in two different applicative instances".
For example: `liftA2 (+) (Just 2) (Just 2) == Just 4`

`<*` and `*>`

There are two more useful operations for applicatives.

1. `<* :: f a -> f b -> f a`
   takes the first applicative and ignores the result of the second if it has one:
   `Just 2 <* Just 3 == Just 2`
   However, it still applies the chaining rules:
   `Just 2 <* Nothing == Nothing`
2. `*> :: f a -> f b -> f b`
   the mirror of `<*` .
   `Just 2 *> Just 3 == Just 3`
   `Just 2 *> Nothing == Nothing`

Why would we want to apply the chaining rules but throw away values?

They're *very* useful

It turns out, `*>` in particular is very useful.

Finally, *finally*, we see something that can help us understand how to write a hello world program in Haskell.

`*>` says basically "chain these two things together but don't try to apply the first one as a function."

We can use this to chain prints:

```
main = putStrLn "hello" *> putStrLn "world"
```

It's not expected that this makes sense yet, I just want to motivate the next module, which is about monads.

Other simple functors and applicatives

There are lots of functors. Remember the `Sum` and `Product` monoids from before? They are functors.

They also implement `Num`, so we can normally just use arithmetic operators on them directly without having to use `<$>`. Example: `Sum 2 + 5 == Sum 7`

But suppose I want to divide the thing inside `Sum 20.0` by 2. This actually fails, because `Num` doesn't guarantee division.

But we can map division by 2 into the `Sum`: `(/2.0) <$> Sum 20.0`

`Product` is the same

Other simple functors and applicatives (2)

These are also `Applicative` instances.

```
Sum (+) <*> pure 10 <*> pure 20 == Sum 30
```

Remember, almost every `Functor` can easily be an applicative. It's not required to do anything interesting when you call `<*>` like `Maybe` or `[]` do, it can just call the function on the value.

These simple "container" types pretty much always do the same thing. They apply the function inside to the value inside.

Questions?

Quiz format

The quiz will consist of 4 questions, each of equal weight.

The questions will involve:

- Defining a typeclass
- Making a datatype an instance of a typeclass, especially `Functor`, `Applicative`, `Semigroup`, `Monoid`, or any other one from this lecture.
- Determining if an implementation follows/can follow certain laws
- Using a typeclass

Let's look at some practice quizzes.

Practice quiz 1

1. (25%) Consider this data type: `data Pair a = Pair a a`
   make it be a `Functor`. The function should apply to both of its `a`s.
2. (25%) What is the result of `(*3) <$> (Pair 7 8)`?
3. (25%) Consider this data type: `data SumPair = SumPair Int Int`
   make it be a `Semigroup` whose binary operation is addition.
4. (25%) Can this type be a `Monoid`? If so, make it one.
   If not, explain why not. Be specific about which law it breaks or other contraint it violates.

Practice quiz 1 answer

```haskell
-- 1.
instance Functor Pair where
    fmap f (Pair x y) = Pair (f x) (f y)
-- 2.: Pair 21 24
-- 3.
instance Semigroup SumPair where
    (SumPair a b) <> (SumPair x y) = SumPair (a + x) (b + y)
-- 4. Yes:
instance Monoid SumPair where
    mempty = SumPair 0 0
```

Practice quiz 2

1. (25%) Consider the `min` operation between two `Integer`s. It returns the more negative of the two. `min 2 3 == 2`. `min -2 0 == -2`. Define a datatype that will be a semigroup wrapper for integers that we want to apply this operation to. I.e., we want `Min 20 <> Min 15 == Min 15`
2. (25%) Make that data type a `Semigroup`. You can assume the funciton `min` exists.
3. (25%) Can we make this data type a `Monoid`? If so, do so. Otherwise, clearly explain why it is impossible.
4. (25%) Can we make this data type a `Functor`? If so, do so. Otherwise, clearly explain why it is impossible.

Practice quiz 2 answer

```haskell
-- 1.
data Min = Min Integer
-- 2.
instance Semigroup Min where
    Min x <> Min y = Min (min x y)
```

3. It is impossible because there is no identity element. Integers are unbounded, so there is no "maximum" integer we could use for 'x' which guarantees `Min x <> Min y == Min y` for any choice of y.
4. No, because a functor must have kind `* -> *`, but Min has kind `*`. If Min were declared as `data Min a = ...`, then we could make it a functor.

Practice quiz 3

1. (25%) Define a data type `Triplet a` which has one constructor which stores three fields of `a`.
2. (25%) Make it a functor. It should apply the function to all 3 of its fields.
3. (25%) Make it an applicative. It should apply each function to its respective value. So `Triplet (+1) (+2) (+3) <*> Triplet 0 0 0 == Triplet 1 2 3`. Define `pure` so that it projects the value to all 3 fields: `pure 7 == Triplet 7 7 7`
4. (25%) Evaluate `liftA2 (*) (Triplet 1 2 3) (Triplet 3 2 1)`.

Practice quiz 3 answers

```haskell
-- 1.
data Triplet a = Triplet a a a deriving Show
-- 2.
instance Functor Triplet where
    fmap f (Triplet x y z) = Triplet (f x) (f y) (f z)
-- 3.
instance Applicative Triplet where
    (Triplet f g h) <*> (Triplet x y z) = Triplet (f x) (g y) (h z)
    pure x = Triplet x x x

liftA2 (*) (Triplet 1 2 3) (Triplet 3 2 1) == Triplet 3 4 3
```

Practice quiz 4

Consider this color-storing data type. It stores `RGB` colors in 3 fields, one for red, one for green, and one for blue: `data Rgb = Rgb Byte Byte Byte deriving (Ord, Eq, Show)`

1. (25%) Make the datatype bounded by making it an instance of the `Bounded a` typeclass. This means you must provide a `minBound :: a` and `maxBound :: a`. Make the minimum value for each of the 3 values 0, and the maximum value 255.
2. (25%) Write a function `clamp` which takes any bounded value (not just `Rgb`) and "clamps" it so that if it's less than `minBound` it returns `minBound`, if it is greater than `maxBound` it returns `maxBound`, and if it's between, returns the unmodified value.
3. (25%) Make it a Semigroup such that `<>` adds the 3 colors but keeps then in bounds (i.e., it clamps the result).
4. (25%) Make it a `Monoid` or explain why you can't.

Practice quiz 4 answers

```haskell
instance Bounded Rgb where
    minBound = Rgb 0 0 0
    maxBound = Rgb 255 255 255

clamp :: (Ord a, Bounded a) => a -> a
clamp x
    | x < minBound = minBound
    | x > maxBound = maxBound
    | otherwise = x

instance Semigroup Rgb where
    (Rgb a b c) <> (Rgb x y z) =
        Rgb (clamp (a + x)) (clamp (b + y)) (clamp (c + z))

instance Monoid Rgb where
    mempty = Rgb 0 0 0
```

Ask an AI

Ask an AI to generate a practice quiz for you like the above! Give it the slides starting with "# Quiz Format" and up to and including this slide. Or give it the whole markdown.

Then, ask it to grade you. I like to use this scale:

1. 0 points off for extremely minor things. Misspellings or missing grouping operators that are clearly intended.
2. 5 points for mistakes that cause the code to fail but are more than just minor mistakes. For example a small type error where it's clear you get the big idea but, e.g., applied the applicative to too many arguments or something.
3. 10 points for bigger mistakes, like type errors that can't work, but there's still "more than half" of the understanding demonstrated.
4. Zero points total if there are several major mistakes or it looks like you're guessing.

Questions?